

Clinical Prompt Engineering: Four-Agent Decision-Support Pipeline

Overview

This portfolio project demonstrates an iterative prompt-engineering workflow for a four-agent clinical decision-support pipeline. Each agent has a focused responsibility, a JSON output contract, Pydantic validation, few-shot examples, safety guardrails, and optional OpenAI-powered smoke tests.

The project is an educational prototype. It is not a medical device, does not replace a licensed clinician, and must not be used to diagnose or treat patients.

Four-Agent Architecture

```
```text
Patient input
 |
 v
Agent 1: Triage -----> urgency, red flags, routing
 |
 v
Agent 2: EHR -----> structured patient-record context
 |
 v
Agent 3: Anamnesis -----> SOCRATES interview and new red flags
 |
 v
Agent 4: Synthesis -----> physician-review clinical summary
```
```

Triage Agent

Classifies urgency from P1 (emergent) to P4 (non-urgent), detects red flags, selects a routing pathway, and requires human escalation for critical cases.

EHR Agent

Returns structured EHR context relevant to the triage pathway. It includes explicit not-found behavior, data-quality flags, medication relevance, and an audit record. The prompt prohibits inventing data for unknown patient identifiers.

Anamnesis Agent

Conducts a structured SOCRATES history interview. It asks one question per turn, avoids repeating information already present in the EHR, watches for new red flags, and stops after a maximum of 12 questions.

Synthesis Agent

Combines triage, EHR, and anamnesis outputs into a draft summary for physician review. It preserves upstream evidence, ranks differential considerations, identifies missing information, and uses hedged rather than definitive diagnostic language.

Technologies Used

- Python
- LangChain
- OpenAI API
- Pydantic

Project Structure

```
```text
|-- Agent1 triage.py
|-- Agent2 ehr.py
|-- Agent3 anamnesis.py
|-- Agent4 synthesis.py
|-- prompt_test_cases.py
|-- run_prompt_tests.py
|-- prompt_iteration_log.md
|-- Prompt_Engineering_Portfolio.pdf
|-- requirements.txt
|-- RELEASE_CHECKLIST.md
|-- README.md
```gitignore
```

Installation

Python 3.11 or newer is recommended.

```
```bash
git clone https://github.com/YOUR_GITHUB_USERNAME/YOUR_REPOSITORY_NAME.git
cd YOUR_REPOSITORY_NAME

python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

On Windows PowerShell, activate the environment with:

```
```powershell
.\.venv\Scripts\Activate.ps1
```

Environment Variable Setup

Create an API key in the OpenAI developer platform, then expose it only through the `OPENAI\_API\_KEY` environment variable. Never commit the key.

macOS or Linux:

```
```bash
export OPENAI_API_KEY="your-api-key"
```

Windows PowerShell:

```
```powershell
$env:OPENAI_API_KEY="your-api-key"
```

The code reads the credential with:

```
```python
os.getenv("OPENAI_API_KEY")
```

OpenAI API billing is separate from a ChatGPT subscription.

## ## Example Usage

Run an individual agent's built-in validation and optional LLM smoke test:

```
```bash
python "Agent1 triage.py"
python "Agent2 ehr.py"
python "Agent3 anamnesis.py"
python "Agent4 synthesis.py"
```

The local schema and validator checks run without an API key. The live LLM section is skipped when `OPENAI\_API\_KEY` is absent.

For programmatic use, load an agent and call its existing run function:

```
```python
import importlib.util
from langchain_openai import ChatOpenAI

spec = importlib.util.spec_from_file_location("triage_agent", "Agent1 triage.py")
triage_agent = importlib.util.module_from_spec(spec)
spec.loader.exec_module(triage_agent)

llm = ChatOpenAI(model="gpt-4o", temperature=0)
result = triage_agent.run_triage(
 llm,
 "I have crushing chest pain and shortness of breath.",
)
print(result.model_dump_json(indent=2))
```

## ## Testing

Compile all Python files:

```
```bash
python -m py_compile \
    "Agent1 triage.py" \
    "Agent2 ehr.py" \
    "Agent3 anamnesis.py" \
    "Agent4 synthesis.py" \
    prompt_test_cases.py \
    run_prompt_tests.py
```

Run the cross-agent live regression suite:

```
```bash
python run_prompt_tests.py
```

Without `OPENAI\_API\_KEY`, the runner exits cleanly and reports that live tests were skipped. With a configured key and API quota, it prints each agent name, test case, input, output, and PASS/FAIL result.

## ## Safety Guardrails

- JSON-only output contracts for predictable downstream processing.
- Pydantic validation for cross-field safety constraints.
- Mandatory human escalation for emergent triage outputs.
- No fabrication of unknown EHR records.
- Immediate escalation when anamnesis discovers a new red flag.
- Evidence-only synthesis using the three upstream outputs.
- Exact physician-review labeling for synthesis output.
- Hedged diagnostic wording and rejection of definitive diagnosis phrases.
- No hard-coded API credentials.

These controls reduce risk but do not establish clinical safety, regulatory compliance, or production readiness.

## ## Prompt Engineering Methodology

Development followed a documented regression cycle:

1. Start with a basic role and task prompt.
2. Add JSON-only output requirements.
3. Add safety rules based on observed failures.
4. enforce critical invariants with Pydantic validators.
5. Retest representative cases with optional LLM smoke tests and evidence-consistency checks.

The detailed version history is in `[`prompt_iteration_log.md`](prompt_iteration_log.md)`, and structured regression cases are in `[`prompt_test_cases.py`](prompt_test_cases.py)`.

## ## Results Summary

The current test layer covers five previously observed or anticipated regressions:

- Chest pain is escalated to P1 rather than under-classified as P2.
- Unknown EHR patient IDs return `not\_found` instead of invented records.
- Anamnesis agent turns contain no more than one question.
- Penicillin allergy evidence is preserved during synthesis.
- Synthesis narratives use hedged, physician-review language.

All Python files compile successfully, local schema checks pass, and the live regression suite is ready to run when an API key and API quota are available.

## ## Responsible Use

This repository contains synthetic examples and prompt-engineering demonstrations. Do not enter protected health information, real patient records, or production credentials. Any clinical deployment would require expert validation, security review, privacy controls, monitoring, and applicable regulatory approval.

